RDF 1.2 N-Quads

A line-based syntax for RDF datasets

W3C Working Draft 22 December 2025



▼ **More details about this document**

This version:
https://www.w3.org/TR/2025/WD-rdf12-n-quads-20251222/

Latest published version:
https://www.w3.org/TR/rdf12-n-quads/

Latest editor's draft:
https://w3c.github.io/rdf-n-quads/spec/

History:
https://www.w3.org/standards/history/rdf12-n-quads/
Commit history

Test suite:
https://w3c.github.io/rdf-tests/rdf/rdf12/rdf-n-quads/

Latest Recommendation:
https://www.w3.org/TR/n-quads

Editors:
Gregg Kellogg (until 2025-09-06), in memoriam
Dominik Tomaszuk

Former editor:
Gavin Carothers (RDF 1.1)

Feedback:
GitHub w3c/rdf-n-quads (pull requests, new issue, open issues)
public-rdf-star-wg@w3.org with subject line `[rdf12-n-quads] … message topic …` (archives)

Copyright © 2012-2025 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Abstract

N-Quads is a line-based, plain text format for encoding an RDF dataset.

RDF 1.2 N-Quads introduces triple terms as a fourth kind of RDF term which can be used as the object of another triple, making it possible to make statements about other statements. RDF 1.2 N-Quads also adds support for directional language-tagged strings.

Status of This Document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the W3C standards and drafts index.

This document is part of the RDF 1.2 document suite. The N-Quads format is a line-based RDF syntax, which is an extension of N-Triples [*RDF12-N-TRIPLES*]. The main distinction is that N-Quads allows the encoding of multiple graphs in a single document representing an RDF Dataset.

This document was published by the RDF & SPARQL Working Group as a Working Draft using the Recommendation track.

Publication as a Working Draft does not imply endorsement by W3C and its Members.

This is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to cite this document as other than a work in progress. Future updates to this upcoming Recommendation may incorporate new features.

This document was produced by a group operating under the W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 18 August 2025 W3C Process Document.

§ Set of Documents

This document is one of eleven RDF 1.2 and twelve SPARQL 1.2 documents produced by the RDF & SPARQL Working Group.

▶ **List of documents**

Table of Contents

Abstract

Status of This Document

Set of Documents

1. Introduction

2. N-Quads Language

2.1 Simple Statements

2.2 Version Announcement

2.3 Triple Terms

2.4 IRIs

2.5 RDF Literals

2.6 RDF Blank Nodes

3. A Canonical form of N-Quads

4. Conformance

4.1 Media Type and Content Encoding

4.1.1 Other Media Types

5. N-Quads Grammar

5.1 White Space

5.2 Comments

5.3 Grammar

5.4 Selected Terminal Literal Strings

6. Parsing
6.1 RDF Term Constructors
6.2 RDF Dataset Construction

A. Privacy Considerations

B. Security Considerations

C. Internet Media Type, File Extension and Macintosh File Type

D. Acknowledgments
D.1 Acknowledgments for RDF 1.1
D.2 Acknowledgments for RDF 1.2

E. Changes between RDF 1.1 and RDF 1.2

F. Index
F.1 Terms defined by this specification
F.2 Terms defined by reference

G. Issue summary

H. References
H.1 Normative references
H.2 Informative references

§ 1. Introduction

This section is non-normative.

This document defines N-Quads, a concrete syntax for RDF [*RDF12-CONCEPTS*], and an extension of N-Triples [*RDF12-N-TRIPLES*]. N-Quads is an easy to parse, line-based, concrete syntax for RDF Datasets [*RDF12-CONCEPTS*].

As with N-Triples, an N-Quads document can contain a single kind of parsing directive for announcing the RDF version of the content. See 2.2 Version Announcement.

N-Quads statements are a sequence of RDF terms representing the subject, predicate, and object of an RDF Triple and an optional graph name identifying a named graph associated with the triple within an RDF dataset, also known as a quad. These may be separated by white space (`spaces`, and/or `tabs`). This sequence is terminated by a period (`.`), optionally followed by white space and/or a comment, and a new line (optional at the end of a document).

EXAMPLE 1: Use of comments in N-Quads

```
<http://one.example/subject1> <http://one.example/predicate1> <http
# or on a line by themselves
_:subject1 <http://an.example/predicate1> "object1" <http://example
_:subject2 <http://an.example/predicate2> "object2" <http://example
```

The RDF dataset represented by an N-Quads document contains exactly each quad matching the N-Quads `statement` production.

§ 2. N-Quads Language

This section is non-normative.

An N-Quads document allows writing down an RDF dataset in a textual form. An RDF dataset is made up of simple statements consisting of a subject, predicate, object, an optional graph name and optional blank lines. Comments may be given after a # that is not part of another lexical token and continue to the end of the line.

§ 2.1 Simple Statements

A simple statement extends the definition of simple triple in [*RDF12-N-TRIPLES*] with an optional named graph.

The simplest statement is a sequence of (subject, predicate, object) terms, forming an RDF triple, and an optional graph name (a blank node identifier or IRI) denoting the named graph in a dataset to which the triple belongs, and terminated by a period (`.`). White space (`spaces`, and/or `tabs`) may surround terms, except where significant as noted in the grammar.

Comments are treated as white space, and may be given after a `#` that is not part of another lexical token and continue to the end of the line.

The graph name can be omitted, in which case the triples are considered part of the default graph of the RDF dataset.

EXAMPLE 2: Simple Statement

```
<http://example.org/#spiderman> <http://www.perceive.net/schemas/re
```

§ 2.2 Version Announcement

The N-Quads language has evolved since its origin, and RDF 1.2 adds new syntax. RDF 1.2 N-Quads introduces the `VERSION` directive along with an optional `version` Media Type parameter. When respectively serializing and parsing N-Quads with new features such as initial text directions or triple terms, authors and parsers can announce and detect the use of the new syntax forms using these directives; similarly, HTTP clients and servers can use the `version` Media Type parameter.

As with N-Triples, the version declaration is case-sensitive.

EXAMPLE 3: Version announcement

```
VERSION "1.2"
<http://example.org/#spiderman> <http://www.perceive.net/schemas/re
```

When providing content over HTTP, servers can announce the version using the optional `version` Media Type parameter:

EXAMPLE 4: HTTP version announcement

```
GET /document.nq HTTP/1.1
Host: example.com
Accept: application/n-quads; version=1.2
```

§ 2.3 Triple Terms

A triple term may be the object of an RDF triple.

A triple term is represented as a `tripleTerm` with `subject`, `predicate`, and `object` preceded by `<<(`, and followed by `)>>`. Note that triple terms may be nested.

EXAMPLE 5: Triple Term

```
_:e38  <ex:familyName>                                        "Smith"
_:anno <http://www.w3.org/1999/02/22-rdf-syntax-ns#reifies> <<( _:e
_:anno <http://example.com/accordingTo>                       _:e22 .
```

§ 2.4 IRIs

As in N-Triples, IRIs may be written only as resolved IRIs. IRIs are preceded by `<` and followed by `>`, and may contain numeric escape sequences (described below). For example `<http://example.org/#green-goblin>`.

§ 2.5 RDF Literals

As in N-Triples, literals are used to identify values such as strings, numbers, dates.

Literals (Grammar production `Literal`) have a lexical form followed by either a language tag (possibly including initial text direction), a datatype IRI, or neither.

The representation of the lexical form consists of an initial delimiter `"`, a sequence of permitted characters or numeric escape sequences or string escape sequences, and a final delimiter.

Literals may not contain the characters `"`, `LF`, or `CR` except in their escaped forms. In addition `\` may not appear in any quoted literal except as part of an escape sequence and a `"` character can only be included in a quoted literal using an escape sequence.

The corresponding lexical form is the characters between the delimiters, after processing any escape sequences. If present, the `LANG_DIR` terminal matches the language tag and optionally the initial text direction. The language tag is preceded by an `@`, and, if present, the initial text direction is separated from the language tag by `--`. If there is no language tag, there may be a datatype IRI, preceded by `^^`. If there is no datatype IRI and no language tag, then it is a simple literal and the datatype is `http://www.w3.org/2001/XMLSchema#string`.

§ 2.6 RDF Blank Nodes

As in N-Triples, RDF blank nodes are expressed as `_:` followed by a blank node label which is a series of name characters. The characters in the label are built upon `PN_CHARS_BASE`,

liberalized as follows:

- The characters _ and the digit characters 0–9 may appear anywhere in a blank node label.
- The character . may appear anywhere except the first or last character.
- The characters ‿, ⁀, ⏑, ⏒, and the combining diacritical marks (U+0300 to U+036F) are permitted anywhere except the first character.

A fresh RDF blank node is allocated for each unique blank node identifier in a document. Repeated use of the same blank node identifier identifies the same blank node.

> EXAMPLE 6: Blank nodes in N-Quads
>
> ```
> _:alice <http://xmlns.com/foaf/0.1/knows> _:bob .
> _:bob   <http://xmlns.com/foaf/0.1/knows> _:alice .
> ```

§ 3. A Canonical form of N-Quads

This section defines a canonical form of N-Quads which has a completely specified layout. The grammar for the language is unchanged.

Canonical N-Quads extends Canonical N-Triples in [*RDF12-N-TRIPLES*] to include `graphLabel`.

While the N-Quads syntax allows choices for the representation and layout of RDF data, the canonical form of N-Quads provides a unique syntactic representation of any quad. Each code point can be represented by only one of UCHAR, ECHAR, or unencoded character, where the relevant production allows for a choice in representation. Each quad is represented entirely on a single line with specified white space.

Canonical N-Quads has the following additional constraints on layout:

- White space *MUST NOT* be used except after `subject`, `predicate`, `object`, and `graphLabel`, any of which *MUST* be a single space.

- A Canonical N-Quads document *MUST NOT* include a VERSION directive.
- Literals with the datatype `http://www.w3.org/2001/XMLSchema#string` *MUST NOT* use the datatype IRI part of the `literal`, and are represented using only STRING_LITERAL_QUOTE.
- HEX *MUST* use only digits ([0–9]) and uppercase letters ([A–F]).
- Alphabetic characters in LANG_DIR *MUST* use only the lowercase letters ([a–z]) with any uppercase letters case mapped to lowercase.
- Within STRING_LITERAL_QUOTE:
  - Characters BS, HT, LF, FF, CR, ", and \ *MUST* be encoded using ECHAR.
  - Characters in the range from U+0000 to U+0007, VT, characters in the range from U+000E to U+001F, DEL, and characters not matching the Char production from [*XML11*] *MUST* be represented by UCHAR using a lowercase \u with 4 HEXes.
  - All characters not required to be represented by ECHAR or UCHAR *MUST* be represented by their native [*UNICODE*] representation.
- The token EOL *MUST* be a single LF.
- The final EOL *MUST* be provided.

§ 4. Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MAY*, *MUST*, *MUST NOT*, and *SHOULD* in this document are to be interpreted as described in BCP 14 [*RFC2119*] [*RFC8174*] when, and only when, they appear in all capitals, as shown here.

This specification defines conformance criteria for:

- N-Quads documents
- Canonical N-Quads documents
- N-Quads parsers

A conforming ***N-Quads document*** is an RDF string that conforms to the grammar and additional constraints defined in 5. N-Quads Grammar, starting with the `nquadsDoc` production. An N-Quads document serializes an RDF dataset.

> NOTE
>
> N-Quads documents do not provide a way of serializing empty graphs that may be part of an RDF dataset.

A conforming ***Canonical N-Quads document*** is an N-Quads document that follows the additional constraints of Canonical N-Quads.

A conforming ***N-Quads parser*** is a system capable of reading N-Quads documents on behalf of an application. It makes the serialized RDF dataset, as defined in 6. Parsing, available to the application, usually through some form of API.

The IRI that identifies the N-Quads language is: `http://www.w3.org/ns/formats/N-Quads`

§ 4.1 Media Type and Content Encoding

The media type of N-Quads is `application/n-quads`. The content encoding of N-Quads is always UTF-8. See N-Quads Media Type for the media type registration form.

§ 4.1.1 Other Media Types

The original specification, N-Quads: Extending N-Triples with Context, proposed the use of media type `text/x-nquads` with an encoding using 7-bit US-ASCII.

§ 5. N-Quads Grammar

An N-Quads document is an RDF string encoded in UTF-8 [*RFC3629*]. Only Unicode scalar values, in the ranges `U+0000` to `U+D7FF` and `U+E000` to `U+10FFFF`, are allowed. This excludes surrogate code points, range `U+D800` to `U+DFFF`.

§ 5.1 White Space

White space (`spaces`, and/or `tabs`) is allowed outside of terminals. Rule names in capitals below indicate where white space is significant.

White space is significant in the production `STRING_LITERAL_QUOTE`.

A ***blank line***, consisting of only white space and/or a comment, may appear wherever a `statement` production is allowed, and is treated as white space.

> NOTE
>
> As with, N-Triples [*RDF12-N-TRIPLES*], N-Quads allows only horizontal white space (`spaces` or `tabs`).

§ 5.2 Comments

Comments in N-Quads start at `#` outside an `IRIREF` or `STRING_LITERAL_QUOTE`, and continue to the end of line — marked by character `CR` or `LF` — or to the end of file, if there is no end of line after the comment marker. Comments are treated as white space.

§ 5.3 Grammar

The EBNF used here is defined in XML 1.0 [*EBNF-NOTATION*].

Escape sequence rules are the same as N-Triples [*RDF12-N-TRIPLES*] and Turtle [*RDF12-TURTLE*]. However, as only the `STRING_LITERAL_QUOTE` production is allowed new lines in

literals *MUST* be escaped.

The 'VERSION' terminal is in single quotes to indicate that it is case-sensitive.

```
[1]  nquadsDoc           ::= statement? (EOL statement)* EOL?
[2]  statement           ::= directive | quad
[3]  directive           ::= versionDirective
[4]  versionDirective    ::= 'VERSION' versionSpecifier
[5]  versionSpecifier    ::= STRING_LITERAL_QUOTE
[6]  quad                ::= subject predicate object graphLabel? '.'
[7]  subject             ::= IRIREF | BLANK_NODE_LABEL
[8]  predicate           ::= IRIREF
[9]  object              ::= IRIREF | BLANK_NODE_LABEL | literal |
                             tripleTerm
[10] graphLabel          ::= IRIREF | BLANK_NODE_LABEL
[11] literal             ::= STRING_LITERAL_QUOTE (('^^' IRIREF) |
                             LANG_DIR)?
[12] tripleTerm          ::= '<<(' subject predicate object ')>>'
```

§ Productions for terminals

```
[14] IRIREF              ::= '<' ([^#x00-#x20<>"{}|^`\] | UCHAR)* '>'
[15] BLANK_NODE_LABEL    ::= '_:' (PN_CHARS_U | [0-9]) ((PN_CHARS | '.')*
                             PN_CHARS)?
[16] LANG_DIR            ::= '@' [a-zA-Z]+ ('-' [a-zA-Z0-9]+)* ('--' [a-zA-
                             Z]+)?
[17] STRING_LITERAL_QUOTE ::= '"' ([^#x22#x5C#x0A#x0D] | ECHAR | UCHAR)* '"'
[18] UCHAR               ::= ('\u' HEX HEX HEX HEX) | ('\U' HEX HEX HEX
                             HEX HEX HEX HEX HEX)
[19] ECHAR               ::= '\' [tbnrf\"']
[20] PN_CHARS_BASE       ::= [A-Z]
                           |   [a-z]
                           |   [#xC0-#xD6]
                           |   [#xD8-#xF6]
                           |   [#xF8-#x02FF]
                           |   [#x0370-#x037D]
                           |   [#x037F-#x1FFF]
```

```
                    |   [#x200C-#x200D]
                    |   [#x2070-#x218F]
                    |   [#x2C00-#x2FEF]
                    |   [#x3001-#xD7FF]
                    |   [#xF900-#xFDCF]

                    |   [#xFDF0-#xFFFD]
                    |   [#x00010000-#x000EFFFF]
[21] PN_CHARS_U     ::= PN_CHARS_BASE | '_'
[22] PN_CHARS       ::= PN_CHARS_U | '-' | [0-9] | #xB7 | [#x0300-
                        #x036F] | [#x203F-#x2040]
[23] HEX            ::= [0-9] | [A-F] | [a-f]
[24] EOL            ::= [#x0D#x0A]+
```

A text version of this grammar is available here.

§ 5.4 Selected Terminal Literal Strings

This document uses some specific terminal literal strings [*EBNF-NOTATION*]. To clarify the Unicode code points used for these terminal literal strings, the following table describes specific characters and sequences used throughout this document.

Code	Glyph	Description
U+0008	BS	Backspace
U+0009	HT	Horizontal tab
U+000A	LF	Line feed
U+000B	VT	Vertical tab
U+000C	FF	Form feed
U+000D	CR	Carriage return
U+0022	"	Quotation mark
U+0023	#	Number sign
U+002D	-	Hyphen

U+002E	.	Full stop
U+0030	0	Digit zero
U+0039	9	Digit nine
U+003B	:	Colon
U+003C	<	Less-than sign
U+003E	>	Greater-than sign
U+0040	@	At sign
U+0041	A	Latin capital letter A
U+0046	F	Latin capital letter F
U+005C	\	Backslash
U+005F	_	Underscore
U+0061	a	Latin small letter A
U+007A	F	Latin small letter Z
U+007F	DEL	Delete
U+00B7	·	Middle dot
U+203F	‿	Undertie
U+2040	⁀	Character tie

Other short terminal literal strings are composed of specific sequences of Unicode characters:

space
: U+0020

<<(
: two concatenated less-than sign characters, each having the code point U+003C, followed by a left parenthesis character, having the code point U+0028

)>>
: a left parenthesis character, having the code point U+0029 followed by two concatenated greater-than sign characters, each having the code point U+003E

: two concatenated greater-than sign characters, each having the code point U+003E

^^
: two concatenated circumflex accent characters, each having the code point U+005E

_:

`_` followed by `:`

`--`

two concatenated `-` characters

§ 6. Parsing

Parsing N-Quads requires a state of two items:

- Map[string -> blank node] `bnodeLabels` — A mapping from string to blank node.
- `xsd:string` *curVersion* – The RDF version used for parsing the document into Quads. If specified as part of a Media Type, the default value for *curVersion* is taken from the `version` parameter. Acceptable values for `version` are defined in 2.1 Version Labels in [*RDF12-CONCEPTS*]. The version announcement is only a hint; this specification does not mandate any parser behavior based on *curVersion*, but a parser *MAY* signal an error or a warning when it encounters a feature that does not match the value of *curVersion*.

§ 6.1 RDF Term Constructors

This table maps productions and lexical tokens to RDF terms or components of RDF terms listed in 6. Parsing:

production	type	procedure
versionSpecifier	literal	The *curVersion* is taken from a literal using the matched RDF string lexical form and `xsd:string` datatype.
BLANK_NODE_LABEL	blank node	The string after `_:`, is a key in bnodeLabels. If there is no corresponding blank node in the map, one is allocated.

IRIREF	IRI	The characters between < and > are taken, with escape sequences unescaped, to form the IRI. The resulting IRI *MUST* comply with the syntactic restrictions of generic IRI syntax, and *SHOULD* conform to section 3.3 of [*RFC3986*] and comply with any narrower restrictions imposed by the corresponding IRI scheme specification.
LANG_DIR	language tag	The characters following the `@` form the language tag and optionally the initial text direction, if the matched characters include `--`. The language tag *MUST* be well-formed according to section 2.2.9 of [*BCP47*]. If present, the initial text direction *MUST* be either `ltr` or `rtl`.
STRING_LITERAL_QUOTE	RDF lexical form	The characters between the outermost quotation marks (") are taken, with escape sequences unescaped, to form the string of the lexical form.
		The literal has a lexical form of the first rule argument, `STRING_LITERAL_QUOTE`, and either a language tag with optional initial text direction from `LANG_DIR` or a datatype IRI of `iri`, depending on which rule matched the input. If the

literal	literal	LANG_DIR rule matched, the language tag and initial text direction are taken from LANG_DIR. If there is no initial text direction, the datatype is `rdf:langString`. If there is a initial text direction, the datatype is `rdf:dirLangString`. If neither LANG_DIR nor datatype IRI match, the literal has a datatype of `xsd:string`.
tripleTerm	triple term	The triple term is composed of the terms constructed from the `subject`, `predicate`, and `object` productions.

NOTE

As processors which detect errors on input can result in datasets which contain fewer triples than are described in the input (including no triples whatsoever), consumers should consider information of any errors signaled when using the resulting dataset, which may be incomplete and/or include ill-typed or ill-formed terms.

§ 6.2 RDF Dataset Construction

An N-Quads document defines an RDF dataset composed of RDF graphs composed of a set of RDF triples. The `statement` production produces a triple defined by the terms constructed for `subject`, `predicate`, and `object`. This RDF triple is added to the graph labeled by the production `graphLabel`, if no `graphLabel` is present the triple is added to the RDF dataset's default graph.

§ A. Privacy Considerations

This section is non-normative.

The N-Quads format is used to express arbitrary application data, which may include the expression of personally identifiable information (PII) or other information which could be considered sensitive. Authors publishing such information are advised to carefully consider the needs and use of publishing such information, as well as the applicable regulations for the regions where the data is expected to be consumed and potentially revealed (e.g., GDPR, CCPA, others), particularly whether authorization measures are needed for access to the data.

§ B. Security Considerations

This section is non-normative.

The `STRING_LITERAL_QUOTE` production allows the use of unescaped control characters. Although this specification does not directly expose this content to an end user, it might be presented through a user agent, which may cause the presented text to be obfuscated due to presentation of such characters.

N-Quads is a general-purpose assertion language; applications may evaluate given data to infer more assertions or to dereference IRIs, invoking the security considerations of the scheme for that IRI. Note in particular, the privacy issues in [*RFC3023*] section 10 for HTTP IRIs. Data obtained from an inaccurate or malicious data source may lead to inaccurate or misleading conclusions, as well as the dereferencing of unintended IRIs. Care must be taken to align the trust in consulted resources with the sensitivity of the intended use of the data; inferences of potential medical treatments would likely require different trust than inferences for trip planning.

The N-Quads language is used to express arbitrary application data; security considerations will vary by domain of use. Security tools and protocols applicable to text (for example, PGP encryption, checksum validation, password-protected compression) may also be used on N-

Quads documents. Security/privacy protocols must be imposed which reflect the sensitivity of the embedded information.

N-Quads can express data which is presented to the user, such as RDF Schema labels. Applications rendering strings retrieved from untrusted N-Quads documents, or using unescaped characters, *SHOULD* use warnings and other appropriate means to limit the possibility that malignant strings might be used to mislead the reader. The security considerations in the media type registration for XML ([*RFC3023*] section 10) provide additional guidance around the expression of arbitrary data and markup.

N-Quads uses IRIs as term identifiers. Applications interpreting data expressed in N-Quads *SHOULD* address the security issues of *Internationalized Resource Identifiers (IRIs)* [*RFC3987*] Section 8, as well as *Uniform Resource Identifier (URI): Generic Syntax* [*RFC3986*] Section 7.

Multiple IRIs may have the same appearance. Characters in different scripts may look similar (for instance, a Cyrillic "o" may appear similar to a Latin "o"). A character followed by combining characters may have the same visual representation as another character (for example, LATIN SMALL LETTER "E" followed by COMBINING ACUTE ACCENT has the same visual representation as LATIN SMALL LETTER "E" WITH ACUTE). Any person or application that is writing or interpreting data in N-Quads must take care to use the IRI that matches the intended semantics, and avoid IRIs that may look similar. Further information about matching visually similar characters can be found in *Unicode Security Considerations* [*UNICODE-SECURITY*] and *Internationalized Resource Identifiers (IRIs)* [*RFC3987*] Section 8.

§ C. Internet Media Type, File Extension and Macintosh File Type

The Internet Media Type (formerly known as MIME Type) for N-Quads is "application/n-quads".

The information that follows has been submitted to the Internet Engineering Steering Group (IESG) for review, approval, and registration with IANA.

Type name:
application

Subtype name:

n-quads

Required parameters:

None

Optional parameters:

`version` — this parameter is optional but *SHOULD* be present when using RDF 1.2-specific features. If present, acceptable values of `version` are defined in 2.1 Version Labels in [*RDF12-CONCEPTS*].

Encoding considerations:

The syntax of N-Quads is expressed over code points in Unicode [*UNICODE*]. The encoding is always UTF-8 [*UTF-8*].

Unicode code points may also be expressed using an `\uXXXX` (`U+0000` to `U+FFFF`) or `\UXXXXXXXX` syntax (for code points up to `U+10FFFF`) where `X` is a hexadecimal digit `[0-9A-F]`

Security considerations:

See B. Security Considerations.

Interoperability considerations:

There are no known interoperability issues.

Published specification:

This specification.

Applications which use this media type:

N-Quads is used widely for representing RDF data. There are implementations available in most common programming languages.

Additional information:

Magic number(s):

None.

File extension(s):

.nq

Macintosh file type code(s):

TEXT

Person & email address to contact for further information:

RDF & SPARQL Working Group <public-rdf-star-wg@w3.org>

Intended usage:

Common

Restrictions on usage:

None

Author(s):

The N-Quads specification is the product of the RDF & SPARQL WG. The W3C reserves change control over this specifications.

§ D. Acknowledgments

This section is non-normative.

§ D.1 Acknowledgments for RDF 1.1

This section is non-normative.

The editor of the RDF 1.1 edition acknowledges valuable contributions from Gregg Kellogg, Andy Seaborne, Eric Prud'hommeaux, Dave Beckett, David Robillard, Gregory Williams, Antoine Zimmermann, Sandro Hawke, Richard Cyganiak, Pat Hayes, Henry S. Thompson, Bob Ferris, Henry Story, Andreas Harth, Lee Feigenbaum, Peter Ansell, Evan Patton and David Booth.

This specification is a product of extensive deliberations by the members of the RDF Working Group chaired by Guus Schreiber and David Wood. It draws upon the earlier specification in *N-Quads: Extending N-Triples with Context*, edited by Richard Cyganiak, Andreas Harth, and Aidan Hogan.

§ D.2 Acknowledgments for RDF 1.2

This section is non-normative.

The editors of the RDF 1.2 edition acknowledge valuable contributions from Andy Seaborne.

In addition to the editors, the following people have contributed to this specification: Andy Seaborne, Denis Ah-Kang, Jim Smart, Peter F. Patel-Schneider, Pierre-Antoine Champin, and Ted Thibodeau Jr

Members of the RDF & SPARQL Working Group Group included Vladimir Alexiev, James Anderson, Amin Anjomshoaa, Julián Arenas-Guerrero, Dörthe Arndt, Bilal Ben Mahria, Erich Bremer, Dan Brickley, Kurt Cagle, Sarven Capadisli, Rémi Ceres, Pierre-Antoine Champin, David Chaves-Fraga, Souripriya Das, Daniil Dobriy, Enrico Franconi, Jeffrey Phillips Freeman, Fabien Gandon, Benjamin Goering, Damien Graux, Adrian Gschwend, Olaf Hartig, Timothée Haudebourg, Ian Horrocks, Gregg Kellogg, Mark Kim, Jose Emilio Labra Gayo, Ora Lassila, Richard Lea, Niklas Lindström, Pasquale Lisena, Thomas Lörtsch, Matthew Nguyen, Peter Patel-Schneider, Thomas Pellissier Tanon, Dave Raggett, Jean-Yves ROSSI, Felix Sasaki, Andy Seaborne, Alan Snyder, Stuart Sutton, Ruben Taelman, Ted Thibodeau Jr, Dominik Tomaszuk, Raphaël Troncy, William Van Woensel, Gregory Williams, Jesse Wright, Achille Zappa, and Antoine Zimmermann.

> EDITOR'S NOTE
>
> Recognize members of the Task Force? Not an easy to find list of contributors.

§ E. Changes between RDF 1.1 and RDF 1.2

This section is non-normative.

- N-Quads is now described as an extension of N-Triples [*RDF12-N-TRIPLES*].
- Add 3. A Canonical form of N-Quads to define the canonical form of N-Quads.
- Better align the use of white space and comments with [*RDF12-TURTLE*].
- Removed language about white space use between terminals that would otherwise be (mis-)recognized, is this can't happen in N-Triples.
- Clarify the use of blank lines, including those composed of white space and/or comments. Comments can appear at the end of a triple before the newline as was already evident from Example 1.

- Create separate subsections of 5. N-Quads Grammar for White space and Comments, better mirroring [*RDF12-TURTLE*].
- Updated the PN_CHARS_U grammar production to be consistent with Turtle. Formerly, PN_CHARS_U included ":" in N-Triples and N-Quads, but not in Turtle nor TriG. PN_CHARS_U is a component of BLANK_NODE_LABEL.
- Adds support for triple terms as described in 2.3 Triple Terms with updates to 6.1 RDF Term Constructors.
- Separated B. Security Considerations from 4.1 Media Type and Content Encoding and updated language.
- Changes the LANGTAG terminal production to LANG_DIR to include an optional initial text direction.
- Added 2.2 Version Announcement and parser state to announce the RDF version associated with the input document.
- Clarify that Unicode surrogates are not legal in N-Quads documents, nor as the value of a Unicode escape sequence.

§ F. Index

This section is non-normative.

§ F.1 Terms defined by this specification

blank line §5.1

Canonical N-Quads document §4.

N-Quads document §4.

N-Quads parser §4.

§ F.2 Terms defined by reference

[*BCP47*] defines the following:

section 2.2.9

[*I18N-GLOSSARY*] defines the following:

case mapped

surrogate code points

Unicode scalar values

[*RDF12-CONCEPTS*] defines the following:

2.1 Version Labels

blank node identifier

datatype IRI

default graph

directional language-tagged strings

graph name

ill-typed

initial text directions

IRI

language tag

lexical form

literals

named graph

object type

predicate

quad

RDF blank nodes

RDF dataset

RDF Datasets

RDF graphs

RDF string

RDF term

simple literal

subject

triple

triple terms

[*RDF12-N-TRIPLES*] defines the following:

Canonical N-Triples

definition of simple triple

[*RDF12-TURTLE*] defines the following:

numeric escape sequences

string escape sequences

[*RFC3986*] defines the following:

resolved

section 3.3

[*RFC3987*] defines the following:

syntactic restrictions

[*XML11*] defines the following:

Char

G. Issue summary

This section is non-normative.

There are no issues listed in this specification.

H. References

H.1 Normative references

[BCP47]

Tags for Identifying Languages. A. Phillips, Ed.; M. Davis, Ed. IETF. September 2009. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc5646

[EBNF-NOTATION]

EBNF Notation. Tim Bray; Jean Paoli; Michael Sperberg-McQueen; Eve Maler; François Yergeau et al. W3C. W3C Recommendation. URL: https://www.w3.org/TR/xml/#sec-notation

[I18N-GLOSSARY]

Internationalization Glossary. Richard Ishida; Addison Phillips. W3C. 17 October 2024. W3C Working Group Note. URL: https://www.w3.org/TR/i18n-glossary/

[RDF12-CONCEPTS]

RDF 1.2 Concepts and Abstract Data Model. Gregg Kellogg; Olaf Hartig; Pierre-Antoine Champin; Andy Seaborne. W3C. 5 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-concepts/

[RDF12-N-TRIPLES]

RDF 1.2 N-Triples. Gregg Kellogg; Dominik Tomaszuk. W3C. 19 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-triples/

[RDF12-TURTLE]

RDF 1.2 Turtle. Gregg Kellogg; Dominik Tomaszuk. W3C. 19 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-turtle/

[RFC2119]

Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc2119

[RFC3629]

UTF-8, a transformation format of ISO 10646. F. Yergeau. IETF. November 2003. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3629

[RFC3986]

Uniform Resource Identifier (URI): Generic Syntax. T. Berners-Lee; R. Fielding; L. Masinter. IETF. January 2005. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3986

[RFC3987]

Internationalized Resource Identifiers (IRIs). M. Duerst; M. Suignard. IETF. January 2005. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc3987

[RFC8174]

Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc8174

[UNICODE]

The Unicode Standard. Unicode Consortium. URL: https://www.unicode.org/versions/latest/

[XML11]

Extensible Markup Language (XML) 1.1 (Second Edition). Tim Bray; Jean Paoli; Michael Sperberg-McQueen; Eve Maler; François Yergeau; John Cowan et al. W3C. 16 August 2006. W3C Recommendation. URL: https://www.w3.org/TR/xml11/

§ H.2 Informative references

[RDF12-INTEROP]

RDF 1.2 Interoperability. Pierre-Antoine Champin. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-interop/spec/

[RDF12-NEW]

What's New in RDF 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-new/spec/

[RDF12-PRIMER]

RDF 1.2 Primer. Niklas Lindström; Pierre-Antoine Champin. W3C. 3 April 2025. DNOTE. URL: https://www.w3.org/TR/rdf12-primer/

[RDF12-SCHEMA]

RDF 1.2 Schema. Dominik Tomaszuk. W3C. 18 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-schema/

[RDF12-SEMANTICS]

RDF 1.2 Semantics. Peter Patel-Schneider; Dörthe Arndt; Enrico Franconi. W3C. 18 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-semantics/

[RDF12-TRIG]

RDF 1.2 TriG. Gregg Kellogg; Dominik Tomaszuk. W3C. 13 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-trig/

[RDF12-XML]

RDF 1.2 XML Syntax. Gregg Kellogg. W3C. 19 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-xml/

[RFC3023]

XML Media Types. M. Murata; S. St. Laurent; D. Kohn. IETF. January 2001. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc3023

[SPARQL12-CONCEPTS]

SPARQL 1.2 Concepts. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-concepts/spec/

[SPARQL12-ENTAILMENT]

SPARQL 1.2 Entailment Regimes. Peter Patel-Schneider. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-entailment/

[SPARQL12-FEDERATED-QUERY]

SPARQL 1.2 Federated Query. Ruben Taelman; Gregory Williams. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-federated-query/

[SPARQL12-GRAPH-STORE-PROTOCOL]

SPARQL 1.2 Graph Store Protocol. Andy Seaborne; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-graph-store-protocol/

[SPARQL12-NEW]

What's New in SPARQL 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-new/spec/

[SPARQL12-PROTOCOL]

SPARQL 1.2 Protocol. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-protocol/

[SPARQL12-QUERY]

SPARQL 1.2 Query Language. Olaf Hartig; Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 21 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-query/

[SPARQL12-RESULTS-CSV-TSV]

SPARQL 1.2 Query Results CSV and TSV Formats. Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-csv-tsv/

[SPARQL12-RESULTS-JSON]

SPARQL 1.2 Query Results JSON Format. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-json/

[SPARQL12-RESULTS-XML]

SPARQL 1.2 Query Results XML Format. Ruben Taelman; Dominik Tomaszuk; Thomas Pellissier Tanon. W3C. 27 December 2024. W3C Working Draft. URL:

https://www.w3.org/TR/sparql12-results-xml/

[SPARQL12-SERVICE-DESCRIPTION]

SPARQL 1.2 Service Description. Ruben Taelman; Gregory Williams. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-service-description/

[SPARQL12-UPDATE]

SPARQL 1.2 Update. Ruben Taelman; Andy Seaborne; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-update/

[UNICODE-SECURITY]

Unicode Security Considerations. Mark Davis; Michel Suignard. Unicode Consortium. 19 September 2014. Unicode Technical Report #36. URL: https://www.unicode.org/reports/tr36/tr36-15.html

